DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: G Hasini
Duration: 1 Week
Total Marks: 50
Submission Date: 05-11-2026

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks Guiding

Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks Guiding

Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks Guiding

Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks Guiding

Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks Guiding

Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c _____d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks Guiding

Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks Guiding

Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks Guiding

Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks Changed

Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding

token-level language identification, or falling back to stemming for out-of-lexicon tokens.

- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS

1.1 Levels of linguistic analysis

Level	Meaning	Hospital-record example	Requirement
Phonology	Sound patterns in spoken language.	A dictated word may be recognised incorrectly by speech recognition.	Not central for typed records; becomes relevant for speech input.
Morphology	Structure of words and grammatical affixes.	fracture, fractures, fractured, fracturing.	Required to connect surface forms with lemmas.
Syntax	Arrangement of words into phrases and sentences.	“No evidence of infection” versus “evidence of infection”.	Useful for interpreting relations and negation; not required for basic entity matching.
Semantics	Meaning of words and sentences.	renal relates to kidney; “negative” changes a test interpretation.	Important for synonym and concept search.
Pragmatics	Meaning in context and speaker intention.	“Rule out pneumonia” is not a confirmed diagnosis.	Needed for safe clinical interpretation.
Discourse	Meaning across multiple sentences or sections.	A discharge note may refer to a diagnosis introduced earlier.	Useful for longitudinal case understanding; beyond the finite-state baseline.

The core assignment directly requires morphology and formal pattern recognition. Tokenisation and entity extraction are also required. Syntax, semantics, pragmatics and discourse can be added in later stages because a finite-state normaliser alone cannot reliably infer the full clinical meaning of a note.

1.2 Three model families

Model family	Suitable hospital task	Reason	Why the other families are less suitable
State machines	Validate a patient identifier or date format.	A finite set of states and transitions can precisely accept or reject regular strings.	A probabilistic model is unnecessary for a fixed format; a large rule system is harder to inspect.
Formal rule systems	Expand known abbreviations and apply spelling/morphology rules.	Rules are explicit, deterministic and auditable.	A state machine alone does not conveniently store all lexical transformations; probability is unnecessary when the mapping is fixed.
Probabilistic models	Rank likely interpretations of misspelt or ambiguous terms.	Context and uncertainty can be estimated from data.	A strict automaton rejects noisy input; hand-written rules cannot cover every context and spelling variation.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “mass” can refer to a physical lump or a quantity; “lead” may be a verb or a metal.

- Morphological ambiguity: “saw” may be the past tense of see or a noun; “-s” may mark plural or third-person singular.
- Segmentation ambiguity: “blood pressure” is a two-word clinical unit, while “12.5mg” may be treated as one token or a number plus unit.
- Abbreviation ambiguity: “MS” can mean multiple sclerosis, morphine sulfate or another local abbreviation depending on context.
- Clinical scope ambiguity: “no fever or cough” may require the negation to apply to both symptoms.

1.4 Search and resource costs

Automaton recognition explores transitions through a finite state space. A DFA processes each input symbol once, giving O(n) time and O(1) working memory apart from the transition table. NFA simulation may track a set of active states, typically O(n·|Q|) in a direct simulation. Regex backtracking can revisit alternatives and may become expensive for poorly designed expressions. The main non-computational costs are expert time for writing patterns, building a lexicon, annotating forms and reviewing clinical abbreviations.

1.5 Proposed pipeline

Clinical records / OCR text / mixed-language notes
↓
[1] Input cleaning and line-joining (regex + OCR correction)
↓
[2] Tokenisation (maximal-munch regular expression)
↓
[3] Entity recognition (12 regex patterns + DFA validators)
↓
[4] Case, abbreviation, date and unit normalisation (rewrite rules)
↓
[5] Morphological analysis (lexicon + finite-state transducer)
↓
[6] Fallback Porter stemming for unknown English forms
↓
[7] Search index: canonical entity + lemma + features
↓
Clinician query → normalise → retrieve ranked records

Input: free-text clinical records and clinician queries. Output: token stream, structured entities, lexical analyses and searchable canonical forms. Each stage is independently testable.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS

2.1 Basic operators

Operator	Meaning	Example	Clinical use
Concatenation	A followed by B.	PT-\d{5}	Patient ID prefix followed by digits.
Disjunction |	Either alternative.	OD|BD|TDS	Dosage frequency.
Kleene star *	Zero or more repetitions.	[A-Z]*	Optional sequence of capital letters.
Plus +	One or more repetitions.	\d+	One or more digits.
Optional ?	Zero or one occurrence.	BP\s*:?	Optional colon after BP.
Character class	One character from a set.	[A-F]	ICD code letter range.
Range	Continuous character interval.	[0-9]	Digit.
Negated class	Any character except listed ones.	[^\n]+	Text until line break.
Anchors	Start/end of string.	^PT-\d{5}$	Validate the complete identifier.
Counters	Bounded repetition.	\d{2,4}	Two to four digits.

2.2 Twelve clinical entity patterns

Entity	Regular expression	Must match	Must not match
Patient identifier	\b(?:PT|PAT)-\d{4,8}\b	PT-10245	PT-12; PT-123456789
Admission date	\b(?:0?[1-9]|[12]\d|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:20\d{2})\b	04/11/2026	44/19/2026
Discharge date	\b(?:0?[1-9]|[12]\d|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:20\d{2})\b	12-11-2026	2026/11/12
Drug + strength	\b[A-Za-z][A-Za-z-]*\s+\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml|IU)\b	Paracetamol 500 mg	500 mg; Paracetamol
Dosage frequency	\b(?:OD|BD|TDS|QID|BID|once daily|twice daily)\b	BD	daily; 5 days
Lab value + unit	\b[A-Za-z][A-Za-z0-9-]*\s*[:=]?\s*\d+(?:\.\d+)?\s*(?:mg/dL|g/dL|mmol/L|U/L|%)\b	Hb 12.5 g/dL	Hb high; 12.5
Blood pressure	\b(?:BP\s*)?\d{2,3}\s*/\s*\d{2,3}\s*mmHg\b	BP 120/80 mmHg	12/8 mmHg
Temperature	\b(?:Temp\s*)?\d{2}(?:\.\d)?\s*°?\s*[CF]\b	Temp 37.0 C	Temp 370 C
ICD code	\b[A-Z]\d{2}(?:\.\d{1,4})?\b	S72.001A	123.45; ABC
Ward and bed	\bWard\s+[A-Z0-9-]+\s*,?\s*Bed\s+\d+[A-Z]?\b	Ward A2, Bed 14	Ward A2
Contact number	\b(?:\+91[-]?)?[6-9]\d{9}\b	9876543210	1234567890
Age	\b(?:Age\s*[:=]?\s*)?\d{1,3}\s*(?:years?|yrs?|y/o)\b	Age: 45 years	Age: 450 years
Symptom duration	\b\d+\s*(?:hours?|days?|weeks?|months?)\b	5 days	five days

The expressions above are screening patterns. Production validators should add calendar validation (for example, rejecting 31 February), clinical range checks, and context checks. The admission and discharge patterns share a date shape; the record section or field label determines which date type it represents.

2.3 Greedy and non-greedy matching

Sentence: "Give 500 mg paracetamol and 250 mg ibuprofen after review."
Greedy: r".*mg" → "Give 500 mg paracetamol and 250 mg"
Non-greedy:r".*?mg" → "Give 500 mg" (first shortest match)

A greedy quantifier consumes as much text as possible while allowing the rest of the pattern to match. A non-greedy quantifier consumes as little as possible. For extracting each dosage, a bounded token pattern such as a drug name followed by a numeric strength is safer than a broad dot-star expression.

2.4 Precedence and grouping

Concatenation binds more tightly than alternation. Therefore, ^A|B$ means (^A) or (B$), not a string that starts with A or B and ends at the end. The intended grouped expression is ^(?:A|B)$, which accepts exactly A or B.

2.5 Error analysis

For each pattern, annotate the gold entity spans and compare them with predicted spans. True positive (TP) is a correct span and type; false positive (FP) is an incorrect extra match; false negative (FN) is a missed gold entity. Precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = 2PR/(P+R). Typical FP causes are broad abbreviations, invalid calendar dates and numbers mistaken for ages. Typical FN causes are OCR substitutions, unusual unit spelling, missing spaces and unlisted abbreviations.

Pattern group	Likely false-positive cause	Likely false-negative cause
Identifiers / ICD	Identifier-like strings in	OCR changes PT to PI or

	narrative text.	confuses 0/O.
Dates / age	Impossible calendar dates or unrelated numbers.	Two-digit years or local date order.
Drug / lab / dosage	A word followed by a number that is not a medication.	Brand names, missing spaces, unfamiliar units.
Vitals / ward / contact	Unlabelled numbers with similar shapes.	Non-standard labels and punctuation.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION

3.1 Token definition and tokenizer

A token is a maximal meaningful unit. The tokenizer preserves hyphenated words, common abbreviations with an optional final full stop, decimal values, dates, alphanumeric clinical codes and a number together with a recognised unit. Punctuation not attached to a token is treated as a boundary.

```
TOKEN_RE = re.compile(
 r"[A-Za-z]+(?:[-'][A-Za-z]+)*\.?|"
 r"\d+(?:\.\d+)?(?:\s?(?:mg|mcg|g|ml|mmol/L|mg/dL|°C|°F))?|"
 r"[A-Z]{1,5}\d{0,6}(?:\.\d+)?|"
 r"\d{1,2}[/-]\d{1,2}[/-]\d{2,4}", re.I)
```

3.2 Maximal-munch examples

Input	Naive whitespace split	Maximal-munch tokenisation
BP 120/80 mmHg	BP | 120/80 | mmHg	BP | 120/80 | mmHg (recognise BP value as a structured span)
Paracetamol 500mg BD	Paracetamol | 500mg | BD	Paracetamol | 500mg | BD
follow-up on 04/11/2026	follow-up | on | 04/11/2026	follow-up | on | 04/11/2026
Hb 12.5 g/dL	Hb | 12.5 | g/dL	Hb | 12.5 g/dL

A whitespace split cannot by itself recognise that “12.5 g/dL” is a single lab measurement or that “04/11/2026” is a date. A maximal-munch scanner tests longer recognised patterns before shorter alternatives and returns the longest valid token at each position.

3.3 Normalisation rules

- Case folding: convert ordinary text to lowercase while preserving a copy of the original for display and audit.
- Abbreviation expansion: BP → blood pressure; HTN → hypertension; DM → diabetes mellitus; SOB → shortness of breath; OD → once daily; BD → twice daily; TDS → three times daily; QID → four times daily; PRN → as needed; IV → intravenous; IM → intramuscular; PO → by mouth; Tab → tablet; Inj → injection; ECG → electrocardiogram; CBC → complete blood count; RFT → renal function test; LFT → liver function test; WBC → white blood cell; RBC → red blood cell; HR → heart rate; RR → respiratory rate; Temp → temperature; SpO2 → oxygen saturation; UTI → urinary tract infection; CAD → coronary artery disease; COPD → chronic obstructive pulmonary disease; CVA → cerebrovascular accident; MI → myocardial infarction; C/O → complains of; H/O → history of; F/H → family history.
- Date canonicalisation: convert day/month/year forms such as 04/11/2026 to ISO-like 2026-11-04 after confirming the local date convention.
- Unit spacing: convert 500mg to 500 mg; standardise equivalent unit spellings only after unit semantics are checked.

3.4 Before-and-after example

BEFORE: PT-10245 admitted on 04/11/2026. C/O fever; BP 120/80 mmHg. Tab Paracetamol 500mg BD x 5 days.
AFTER: pt-10245 admitted on 2026-11-04. complains of fever; blood pressure 120/80 mmhg. tablet paracetamol 500 mg twice daily x 5 days.

3.5 Irreversible normalisation and limits

Case folding removes original capitalisation; abbreviation expansion removes the exact shorthand chosen by the author; unit conversion may remove the original unit representation; punctuation stripping may destroy section or sentence boundaries. These operations are acceptable for retrieval when the original text is retained alongside the normalised index. They are not acceptable if the transformed text replaces the legal or clinical source record.

A classical regular expression cannot recognise arbitrary balanced nested parentheses because that language is not regular. It also cannot reliably decide that “MS” means multiple sclerosis rather than morphine sulfate without contextual evidence. Such requirements need a parser, lexicon/context model or a probabilistic component.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE

4.1 Formal definition

A finite automaton is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the initial state and F is the set of accepting states. A DFA has exactly one next state for each state-symbol pair; an NFA may have multiple next states and ε-transitions.

4.2 DFA designs

The following compact state descriptions define four DFA validators. Each validator can be expanded into a complete transition table by grouping all characters not listed into a dead state.

Validator	State path / transition description	Acceptance condition
Patient ID PT-12345	q0 --P→ q1 --T→ q2 --'-'→ q3 --digit→ q4 --digit→ q5 --digit→ q6 --digit→ q7 --digit→ q8*	Accept after 4–8 digits; q8 is accepting and digit loops up to the maximum.
Contact number 9876543210	q0 --[6-9]→ q1 --digit→ q2 ... --digit→ q10*	Exactly ten digits; q10 accepting; any extra character goes to dead state.
ICD-like code S72.001	q0 --[A-Z]→ q1 --digit→ q2 --digit→ q3; optional '.' then 1–4 digits	Accept after two digits or after a dot and 1–4 digits.
Blood pressure 120/80 mmHg	q0 --2/3 digits→ q1 --'/'→ q2 --2/3 digits→ q3 --space→ q4 --mmHg→ q5*	Accept only after the unit suffix; spaces can be optional in implementation.

4.3 NFA with optional identifier prefix

Consider an identifier that may begin with PT- or PAT- and then contains 4–6 digits. The NFA branches after the initial P: one branch consumes T-, while another consumes A, T-. This is convenient to describe as a nondeterministic choice.

q0 --P--> q1
q1 --T--> q2 --'-'--> q5
q1 --A--> q3 --T--> q4 --'-'--> q5
q5 --digit--> q6 --digit--> q7 --digit--> q8 --digit--> q9*
q9 --digit--> q10* (optional fifth/sixth digit paths are accepting)

Accepted trace for PT-1234: {q0} → {q1} → {q2} → {q5} → {q6} → {q7} → {q8} → {q9}; the final state is accepting. Rejected trace for PX-1234: after reading P the machine reaches q1, but X has no transition, so the active-state set becomes empty and the string is rejected.

4.4 Subset construction and minimisation

Subset construction converts an NFA to a DFA by making each DFA state a set of NFA states. Start with ε-closure({q0}); for each input symbol, take the union of reachable NFA states and then its ε-closure. A subset is accepting if it contains at least one NFA final state. Equivalent DFA states can then be merged using partition refinement: initially separate final and non-final states, then repeatedly split groups whose transitions lead to different groups.

Subset	Input P	Input A	Input T	Input '-'	Input digit	Accept?

state						
{q0}	{q1}	∅	∅	∅	∅	No
{q1}	∅	{q3}	{q2}	∅	∅	No
{q2}	∅	∅	∅	{q5}	∅	No
{q3}	∅	∅	{q4}	∅	∅	No
{q4}	∅	∅	∅	{q5}	∅	No
{q5}	∅	∅	∅	∅	{q6}	No
{q6}	∅	∅	∅	∅	{q7}	No
{q7}	∅	∅	∅	∅	{q8}	No
{q8}	∅	∅	∅	∅	{q9}	Yes

The table is a teaching trace for the two prefix branches; a production subset table must include every digit-count state through the permitted maximum and a dead state for every missing transition. State minimisation must be performed on the completed DFA, not on an incomplete sketch. The minimised count should be reported from the generated machine.

4.5 Thompson construction and reverse conversion

Thompson's construction converts a regular expression into an ε-NFA. For the expression PT-[0-9]{4}, create a fragment for P, concatenate T and '-', then concatenate four digit fragments. Alternation is represented by a new split state with ε-edges to each alternative; Kleene star uses ε-edges to allow zero or repeated traversals. Conversely, state elimination can derive a regular expression: eliminate intermediate states while updating edge labels with $R_{ij} \cup R_{ik}(R_{kk})^*R_{kj}$. The accepted language remains unchanged.

4.6 Pumping lemma

The pumping lemma states that for every regular language L there is a pumping length p such that any string s in L with $|s| \geq p$ can be written s = xyz, where $|xy| \leq p$, $|y| > 0$ and $xy^iz \in L$ for every $i \geq 0$. Consider $L = \{ ('^n)'^n : n \geq 0 \}$, representing exactly balanced runs of opening and closing brackets. Choose $s = ('^p)'^p$. Since $|xy| \leq p$, y contains only opening brackets. Pumping y with i = 0 produces fewer opening than closing brackets, so the result is not in L. Contradiction: L is not regular. Therefore, arbitrary balanced nesting cannot be validated by a finite automaton.

TASK 5 – INFLECTIONAL MORPHOLOGY

5.1 Key terms and distinction

A morpheme is the smallest unit that carries meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme placed before or after a stem. Inflection changes grammatical features without normally changing the word class or core dictionary meaning. A paradigm is the set of forms belonging to one lexeme.

Criterion	Inflection	Derivation
Category	Usually preserves word class.	May change word class.
Meaning	Adds grammatical information such as number or tense.	Creates a new lexical meaning.
Productivity	Often highly regular.	Varies by affix and lexical family.
Position	Typically outside derivational material.	Often closer to the root than inflection.

5.2 Clinical inflectional paradigms

Lexeme	Plural / noun forms	Verb forms	Notes
fracture	fracture, fractures	fracture, fractured, fracturing	Regular noun plural and verb inflection.
infection	infection, infections	infect, infects, infected, infecting	Noun and related verb are distinct lexemes.
treat	—	treat, treats, treated, treating	Regular verb paradigm.
wound	wound, wounds	wound, wounds, wounded, wounding	Same spelling can be noun or verb.
patient	patient, patients	—	Noun number;

			adjective use also possible.

5.3 Orthographic rules

Rule	Implementation	Example	Over-generation counter-example
Drop final e before -ing	If stem ends in silent e, remove e before -ing.	treat + ing → treating	see + ing is seeing, not *sing.
Double final consonant	For eligible stressed CVC stems, double final consonant before -ed/-ing.	admit → admitted	open → opened, not *openned.
y to i	Consonant + y becomes i before -es/-ed.	injury → injuries	play → played, not *plaied.
Sibilant -es	Add -es after s, x, z, ch, sh.	dose → doses	day → days, not *dayes.
Regular -s	Add -s to most nouns/verbs.	patient → patients	child → children is irregular.

5.4 Irregular forms

Surface form	Lemma	Reason
children	child	Suppletive/irregular plural.
men	man	Vowel change and irregular plural.
women	woman	Vowel change and irregular plural.
feet	foot	Vowel change and irregular plural.
teeth	tooth	Vowel change and irregular plural.
was / were	be	Suppletive past-tense forms.
went / gone	go	Suppletive verb forms.
better	good	Irregular comparative.

Irregular forms must be listed because a simple productive suffix rule cannot derive their spelling and sometimes their stem. A lexicon provides the required mapping.

5.5 Measuring inflection

Type-to-lemma ratio = number of distinct surface word types divided by number of distinct lemmas. A lower ratio after lemmatisation indicates that multiple inflected forms have been grouped. To estimate query failures explained by inflection, manually label failed queries whose missing documents contain only an inflected variant of the query lemma. Divide those failures by all failed queries. These measurements require the specified corpus and query judgements; they cannot be inferred from the assignment brief alone.

TASK 6 – DERIVATIONAL MORPHOLOGY

6.1 Derivational affix inventory

Affix	Function / category effect	Clinical example
-itis	N → N; inflammation	arthritis, nephritis
-ectomy	N → N; removal procedure	nephrectomy
-ology	N → N; field of study	cardiology
-osis	N → N; condition	fibrosis
-aemia	N → N; blood condition	anaemia
-ation	V → N; process/result	inflammation (historical formation), administration
-ment	V → N; result	treatment

-able	V → ADJ; capability	treatable
-ive	V/N → ADJ; quality	protective
-al	N → ADJ; relation	renal
-ic	N → ADJ; relation	cardiac
-ist	N → N; specialist/person	cardiologist
-ian	N → N/ADJ; person/relation	paediatrician
hyper-	prefix; excessive/high	hyperglycaemia
hypo-	prefix; low/under	hypotension
nephro-	combining form; kidney	nephrology
cardio-	combining form; heart	cardiovascular
re-	V → V; again	reoperate
un-	negative/reversative	unhealed

6.2 Derivational families

Root / base	Family	Attachment order
kidney / nephro	nephro + logy → nephrology; nephro + ectomy → nephrectomy	Combining form + derivational suffix.
cardi	cardio + logy → cardiology; cardio + logist → cardiologist	Root/combining form + suffix.
infect	infect + ion → infection; infect + ed → infected	Derivation first for infection; inflection for infected.
treat	treat + ment → treatment; treat + able → treatable; re + treat → retreat	Prefix/suffix attaches to verbal base.
fracture	fracture + s → fractures; fracture + ed → fractured	Inflection attaches after lexical base.
pressure	hyper + tension → hypertension; hypo + tension → hypotension	Prefix changes the clinical concept.

Impossible or highly restricted orders include *-itis + nephro- when the intended form is nephritis (the combining form and suffix have a conventional order), and *treat-ed-ment when the intended noun is treatment (the derivational noun suffix attaches to the base, not after a past-tense inflection).

6.3 Semantic non-compositionality and search impact

A derived term may have a specialised meaning that is not the simple sum of its parts. “Nephrectomy” is a specific surgical removal of a kidney, not merely any kidney-related action. “Inflammation” is a clinical condition, and “treatment” may refer to a plan or intervention. Aggressive stemming may collapse distinct clinical concepts or create misleading matches. For example, “renal” and “nephrectomy” are related to the kidney but are not interchangeable; “infectious” and “infection” are related but play different grammatical and clinical roles.

6.4 Productivity and conflation decision

Productivity should be measured by counting distinct derived types and total token occurrences for each affix. An affix is productive when it forms new, interpretable words across multiple bases; it is fossilised when it occurs in a small fixed set of lexicalised forms. Without the required corpus, no empirical productivity counts can be honestly reported.

Search relationship	Benefit / risk	Decision
fracture ↔ fractures	Plural variation should retrieve the same concept.	Conflate by lemmatisation.
infect ↔ infected	Tense/aspect variation often supports retrieval.	Conflate cautiously while preserving features.
renal ↔ nephrectomy	Related concepts but different clinical meanings.	Do not automatically conflate.
hypertension ↔ hypotension	Opposite clinical states share a suffix/root.	Never conflate as equivalent.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING

7.1 Finite-state transducer

A finite-state transducer (FST) is an automaton with an input tape and an output tape. The lexical tape contains the lemma and grammatical features; the surface tape contains the written word. A transition can consume a lexical symbol and emit a surface symbol. The same relation can be traversed from surface to lexical analysis (parsing) or from lexical input to surface form (generation).

Lexical tape: fracture +N +PL
↓ FST relation
Surface tape: fractures
Analysis: fracture +N +PL

7.2 Continuation-class design

Class	Lexical entries	Permitted continuation
NOUN	fracture, patient, infection, wound	+SG → zero; +PL → -s / -es; irregular plural lookup.
VERB	treat, infect, admit, fracture	+BASE → zero; +3SG → -s/-es; +PAST → -ed; +PROG → -ing.
ADJECTIVE	normal, abnormal, acute, chronic	+POS → zero; +COMP → -er; +SUPER → -est (lexical restrictions apply).
DERIVED	nephro-, cardio-, treat, infect	Derivational suffix/prefix paths; then optional inflection where licensed.

7.3 Hand-traced analyses

Surface form	Consumed surface ending	Lexical output	Analysis
fractures	-s	fracture	fracture +N +PL (or +V +3SG; context resolves)
fractured	-ed	fracture	fracture +V +PAST / +V +PASTPART
fracturing	-ing	fracture	fracture +V +PROG
patients	-s	patient	patient +N +PL
infections	-s	infection	infection +N +PL

A simplified trace begins at the lexical stem path, follows the continuation class for the part of speech, consumes the matching suffix on the surface tape and emits the grammatical feature on the lexical tape. In an implementation, each transition is recorded as (state, lexical symbol:surface symbol, next state).

7.4 Generation and ambiguity

Generation starts with fracture +N +PL and traverses the same relation in the opposite direction to produce fractures. Parsing starts with fractures and returns fracture +N +PL. This is one bidirectional relation, not two separately authored rule sets.

Ambiguity examples: “wound” can be a noun or the past tense of “wind”; “saw” can be the noun saw or the past tense of see. The FST should return all licensed analyses. A downstream POS tagger or clinical context component chooses the most likely interpretation while retaining alternatives when confidence is low.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL

MORPHOLOGY

8.1 Why a lexicon transducer is not enough

A lexicon FST that simply concatenates morphemes may generate incorrect spellings. For example, admit + ed should become admitted rather than *admited; injury + s should become injuries rather than *injurys; treat + ing should become treating rather than *treateing. Orthographic rules must operate at morpheme boundaries.

8.2 Rewrite rules

Rule	Rewrite notation	Example
Final e deletion	e → ∅ / stem __ +ing	treat + ing → treating
Consonant doubling	C → CC / stressed CVC stem __ +ed/+ing	admit + ed → admitted
y-to-i	y → i / consonant __ +es	injury + es → injuries
Sibilant insertion	∅ → e / sibilant __ +s	dose + s → doses
Plural allomorph	s → es / sibilant __	match + s → matches

8.3 Cascade and two-level model

Lexicon FST → Morpheme boundary representation →
Rule FST 1 (e deletion) → Rule FST 2 (y-to-i) →
Rule FST 3 (doubling) → Rule FST 4 (sibilant -es) → Surface form

A sequential cascade applies rules in a specified order and is easy to inspect and debug. A two-level (Koskenniemi-style) model represents lexical and surface correspondences in parallel constraints, reducing dependence on a long sequential cascade. Cascades are often simpler for small teaching systems; two-level rules can be more elegant when many interacting spelling correspondences must be maintained.

8.4 Test set

Lexical input	Expected surface	Rule exercised
treat + ing	treating	e deletion
admit + ed	admitted	consonant doubling
injury + es	injuries	y-to-i
dose + s	doses	sibilant plural
match + s	matches	sibilant plural
patient + s	patients	regular plural
fracture + s	fractures	regular plural
study + ed	studied	y-to-i
stop + ed	stopped	doubling
operate + ing	operating	e deletion
box + s	boxes	sibilant plural
wish + es	wishes	sibilant plural
play + ed	played	y rule blocked after vowel
open + ed	opened	doubling blocked
see + ing	seeing	special e sequence
fix + es	fixes	sibilant plural
injury + s	injuries	y-to-i allomorph
admit + ing	admitting	doubling
care + able	careable / context-dependent	lexical restriction
child + plural	children	irregular lexicon

The table is a rule-focused test suite. Forms with lexical restrictions or irregular morphology are handled by explicit lexicon entries rather than overgeneralised spelling rules.

TASK 9 – THE PORTER STEMMER

9.1 Definitions

The Porter stemmer is a rule-based suffix-stripping algorithm. In its classical formulation, consonant/vowel classification is defined over a word; y behaves as a consonant at the beginning and alternates according to the preceding character. The measure m is the number of vowel-consonant (VC) sequences in the stem. The condition *v means the stem contains a vowel; *d means the stem ends in a double consonant; *o means the stem ends in consonant-vowel-consonant where the final consonant is not w, x or y.

Word	Illustrative VC pattern	m (approx.)
fracture	CCVCVCV	2
infection	VCVCVCVC	3
nephrectomy	CVC CCVCVCV (simplified)	3
admitted	VC CVCVC	2
abnormality	VC CVCVCVC	3
patients	CVCVCC	2

The exact m calculation depends on the stem boundary and the Porter treatment of y. The implementation in the appendix includes a transparent educational approximation; for a formal experiment, use a verified Porter implementation such as NLTK's PorterStemmer and document the library version.

9.2 Five algorithm stages

Stage	Main operations	Clinical illustration
Step 1a	Plural reductions: -sses, -ies, -ss, -s.	fractures → fracture (simple plural handling).
Step 1b	Past/progressive suffix removal under vowel/measure conditions.	treated → treat; treating → treat.
Step 1c	Terminal y becomes i when the stem contains a consonant.	injury → injuri (stemmer output may not be a dictionary word).
Step 2 and 3	Replace longer derivational suffixes with shorter forms.	administration may be reduced toward administr/administrate depending on rules.
Step 4 and 5	Remove selected suffixes when m is large enough; remove final e or one l in ll.	A resulting stem is not guaranteed to be a valid word.

9.3 Stemming versus lemmatisation

Input	Porter-style stem (illustrative)	Lemma / FST analysis
fractures	fractur	fracture +N +PL
fractured	fractur	fracture +V +PAST
fracturing	fractur	fracture +V +PROG
patients	patient	patient +N +PL
children	children (unless irregular lexicon added)	child +N +PL
better	better	good +ADJ +COMP

Stemming mechanically removes suffixes and may produce a non-word. Lemmatisation uses lexical and grammatical knowledge to return a dictionary lemma. The FST is therefore more linguistically precise but requires a maintained lexicon and rules.

9.4 Error types

Over-stemming occurs when unrelated words are reduced to the same stem, such as clinical terms whose suffixes distinguish different conditions or procedures. Under-stemming occurs when related forms remain separate because the algorithm does not recognise an irregular or derivational relationship. Rates must be measured over the 200 hand-annotated forms. For each pair, record whether the conflation is correct, incorrect or missed.

9.5 Retrieval experiment

Condition	Precision	Recall	F1	Interpretation

No normalisation	To measure	To measure	To calculate	Exact matching baseline.
Porter stemming	To measure	To measure	To calculate	Often increases recall but may reduce precision.
Full morphological analysis	To measure	To measure	To calculate	Expected to preserve features and reduce unsafe conflation; lexicon coverage limits recall.

Do not insert fabricated numerical values in this table. Run the 40-query evaluation and fill each metric using the relevance judgements. Report macro-averaged metrics across queries and also the number of queries with zero retrieved relevant documents.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE

10.1 Implementation

The attached Python file implements a compact educational prototype using the standard library: entity regular expressions, a tokenizer, abbreviation expansion, date and unit normalisation, a small lexicon, inflectional analysis and a Porter-style suffix stemmer. It prints a sample execution and is intended to be extended for the full assignment resource sizes.

10.2 Sample input and expected behaviour

INPUT:
PT-10245 admitted on 04/11/2026 with fractures and fever.
BP 120/80 mmHg, Temp 37.0 C. Hb 12.5 g/dL.
Paracetamol 500mg BD for 5 days. Ward A2, Bed 14.

EXPECTED DETECTIONS:
patient_id = PT-10245
date = 04/11/2026
blood_pressure = 120/80 mmHg
temperature = 37.0 C
lab_value = Hb 12.5 g/dL
drug_strength = Paracetamol 500mg
frequency = BD
ward_bed = Ward A2, Bed 14

10.3 Test strategy

- Positive tests: one valid example for every entity pattern and every automaton.
- Negative tests: malformed identifier, impossible date, invalid contact number, incomplete blood-pressure value and unrecognised dosage frequency.
- Morphology tests: regular plural, past tense, progressive form, adjective comparison and each listed irregular form.
- Ambiguity tests: wound and saw must return more than one lexical analysis when context is absent.
- Regression tests: compare program output with the hand traces in Tasks 4, 7 and 9.

10.4 Evaluation table template

Pipeline stage	**Items processed**	**Accepted / analysed**	**Rejected / unknown**	**Runtime (s)**
Tokenisation	[fill]	[fill]	[fill]	[fill]
Entity patterns	[fill]	[fill]	[fill]	[fill]
DFA validation	[fill]	[fill]	[fill]	[fill]
Morphological FST	[fill]	[fill]	[fill]	[fill]
Porter fallback	[fill]	[fill]	[fill]	[fill]

10.5 Plot suggestion

Create a line plot of processing time against record length (for example 100, 250, 500, 1,000 and 2,000 tokens). Alternatively, plot morphological analysis coverage against lexicon size. The plot must be generated from actual program runs and should include labelled axes, units and the machine/software environment.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN

Criterion	No normalisation	Porter stemming	Finite-state morphology
Recall	Low for inflected variants.	Usually higher for suffix variants.	High for covered lexical families and licensed forms.
Precision	Can be high for exact matches but misses variants.	May fall due to over-stemming.	Can be high when lexicon and rules are accurate.
Coverage	No linguistic resources required.	Broad English suffix coverage.	Limited by lexicon and rule coverage.
Runtime	Very low.	Low and approximately linear in token length.	Low-to-moderate; depends on transducer size and ambiguity.
Maintenance	Low.	Low-to-moderate.	Higher: lexicon, affixes and rules require expert review.

Recommendation: deploy a hybrid, auditable architecture. Use regex and DFA for structured fields, finite-state morphology for known clinical English forms, and a conservative Porter fallback only for unknown English tokens. Preserve original text and all analyses. Do not conflate clinically opposed or merely related concepts such as hypertension and hypotension. Add contextual ranking for ambiguous abbreviations and derived terms.

The best design is not necessarily the one with the highest recall. In a hospital, a misleading match can be more harmful than a missed related record. Precision, provenance, explainability and clinician review are therefore essential. A learned model becomes economical when the volume and diversity of new vocabulary, code-mixing and spelling noise make manual lexicon maintenance more expensive than collecting representative labelled data.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

A. Impact analysis

The new inputs contain OCR character substitutions, broken words at line endings, missing diacritics, out-of-lexicon terms and Romanised regional-language tokens mixed with English. The original assumptions of clean typed text, closed vocabulary, English-only morphology and exact accept/reject matching no longer hold.

New input property	Original component affected	Expected failure
0/O, 1/l, rn/m confusion	Regex and DFA validators	Valid identifiers, dates and drug names may be rejected.
Line-break hyphenation	Tokenizer	One word may be split into two tokens.
Missing diacritics	Lexicon / language identification	Regional-language forms may not match canonical entries.
Out-of-lexicon words	FST	No lexical analysis is returned.
Tamil-English code-mixing	English morphology	English-only rules misanalyse Tamil suffixes or particles.

B. Evaluation of original design

Three representative noisy records illustrate the failure points. These are constructed test cases, not real patient records.

Record type	Input example	First failure in original pipeline
OCR	PT-1O245: fracture of femur; BP 12O/8O mmHg.	The identifier and blood-pressure regex reject letter O in place of zero.
Code-mixed	patient ku fever irundhuchu, paracetamol 500mg kuduthen.	English-only morphological analyser cannot analyse ku, irundhuchu or kuduthen.
OCR + mixed	PATI0NT has fevcr; paracetamol 5O0mg kuduthen.	OCR noise causes entity extraction failure; regional-language tokens remain out of lexicon.

The exact degradation in precision, recall, analyser coverage and retrieval F1 must be measured by running the same labelled records through the baseline and revised pipelines. A character-level edit-distance matcher or weighted transducer is more appropriate for OCR variants than adding every possible typo as a separate regex. A character n-gram language identifier can help route mixed-language tokens to the correct analyser.

C. Adaptation / redesign

Decision: MODIFY the original finite-state pipeline rather than replace it. The finite-state layer remains useful for exact structured patterns and regular morphology, but it must be preceded by noise-tolerant processing and supplemented with language-aware analysis.

- Add an OCR correction layer using weighted edit distance and a confusion table for 0/O, 1/l/I and rn/m. Apply corrections only when the candidate is supported by field context; retain the raw OCR string.
- Join line-break hyphenation when a line ends with a hyphen and the next line continues a word. Keep a provenance flag recording the merge.
- Add token-level language identification using character n-grams and a mixed-language label rather than forcing every token into English.
- Extend the lexicon with validated Romanised Tamil clinical phrases and common variants. Store language and transliteration variants separately.
- Add a fallback path for unknown English tokens using conservative stemming, while unknown non-English tokens are preserved and flagged for annotation.
- Use weighted automata to rank near-matches instead of accepting every edit-distance candidate as correct.
- Retain the original entity patterns for high-confidence exact matches and add tolerant alternatives only for selected OCR-sensitive fields.

D. Evidence-based defence

This modification is justified because exact regex and DFA validation is reliable on clean structured text but brittle under OCR substitutions. Morphological FSTs provide transparent analyses for known English and regional-language entries, while a weighted correction layer handles uncertain surface spellings. Token-level language identification prevents English rules from being incorrectly applied to Tamil morphology. The design also controls risk by preserving original text, returning confidence and alternatives, and requiring human review for uncertain clinical entities.

Remaining limitations: a finite-state layer cannot fully understand clinical intent, resolve every abbreviation from context, interpret complex negation and temporal relations, or guarantee correctness for unseen language varieties. OCR correction can introduce a wrong word if applied without context. Code-mixed text may contain spelling variation that no finite lexicon can exhaustively cover. These limitations require contextual NLP and clinician validation.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**